



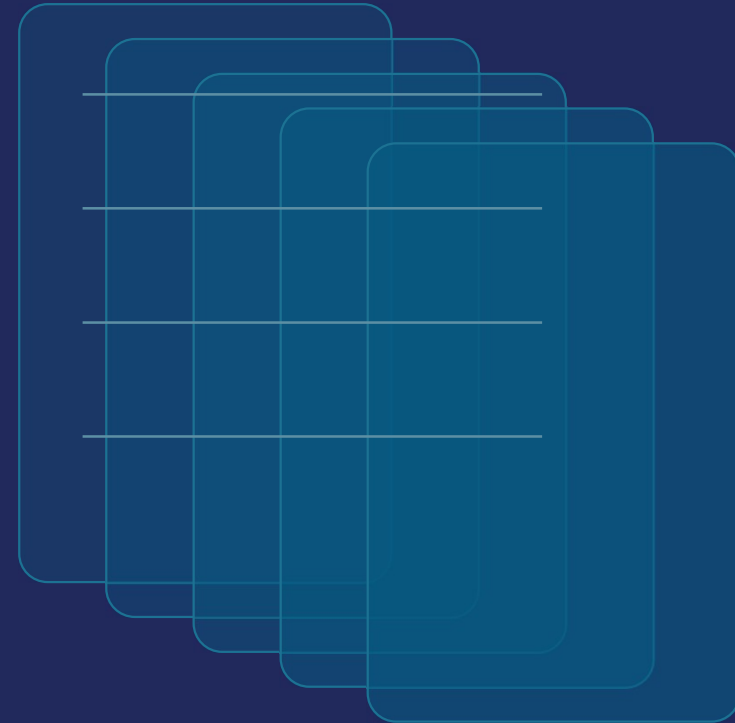
KUBERNETES TRAINING

Module 2:

Kubernetes Architecture & kubectl

Understand the cluster control plane, worker-node runtime, and the CLI path into Kubernetes.

2026 MODERNIZED EDITION



From containers to orchestration

MODULE 1 RECAP

- | | | |
|----------|-------------------|---|
| 1 | Image | Read-only package of app + dependencies |
| 2 | Container | Running instance created from an image |
| 3 | Docker CLI | Build, run, inspect, stop, remove |
| 4 | Registry | Shared source for images |

MODULE 2 ROADMAP

- | | |
|--------------------------------|----------------------------------|
| 01 Why Kubernetes? | 02 Control plane |
| 03 Worker node | 04 Full cluster |
| 05 kubectl + kubeconfig | 06 Operate + troubleshoot |



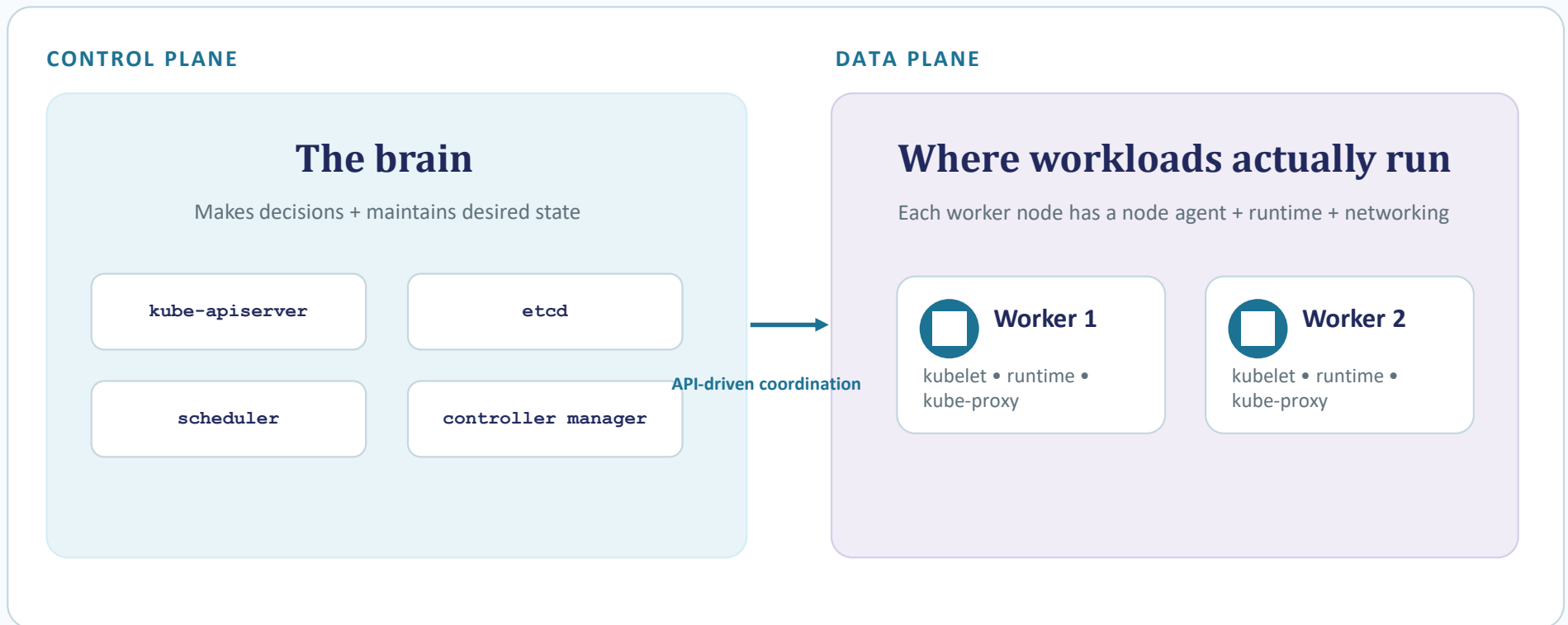
Why Kubernetes?

Docker is great at packaging and running containers. Production asks a bigger question:

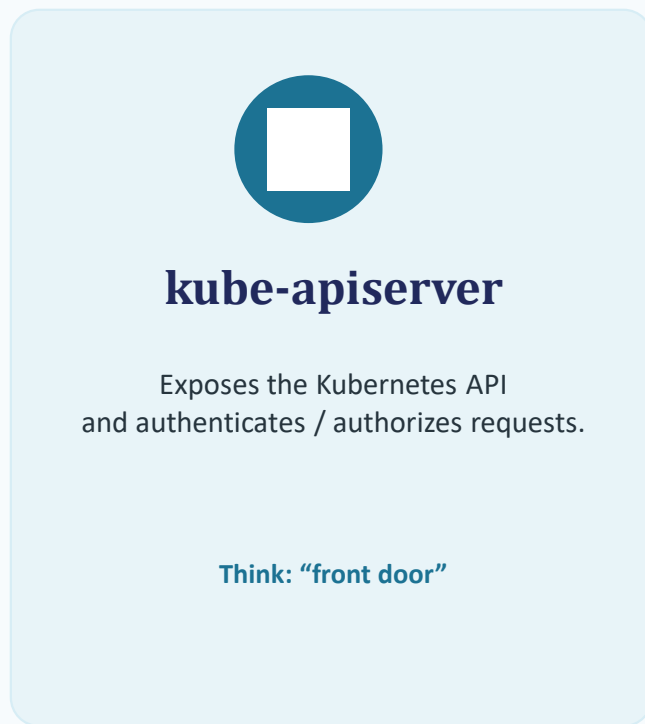
“Who runs what, where, how many, and what happens when something fails?”

Scheduling	→	Manual placement → Kubernetes chooses suitable nodes
Load balancing	→	No built-in Docker solution → Services route traffic
Scaling	→	Manual adjustments → horizontal scaling can be automated
Self-healing	→	Detect + restart unhealthy workloads
Discovery + config	→	DNS, ConfigMaps, Secrets decouple app configuration

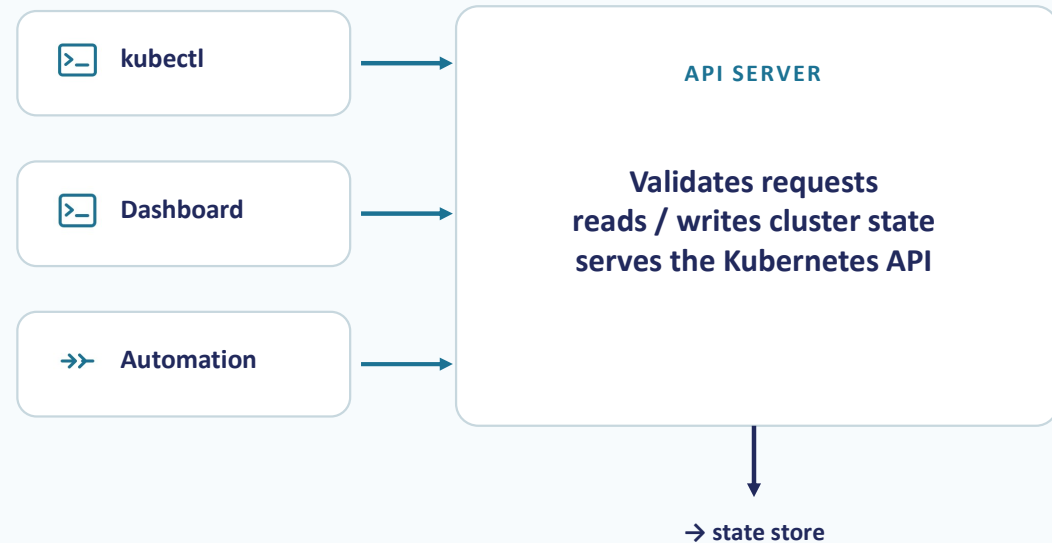
Kubernetes architecture — 10,000-foot view



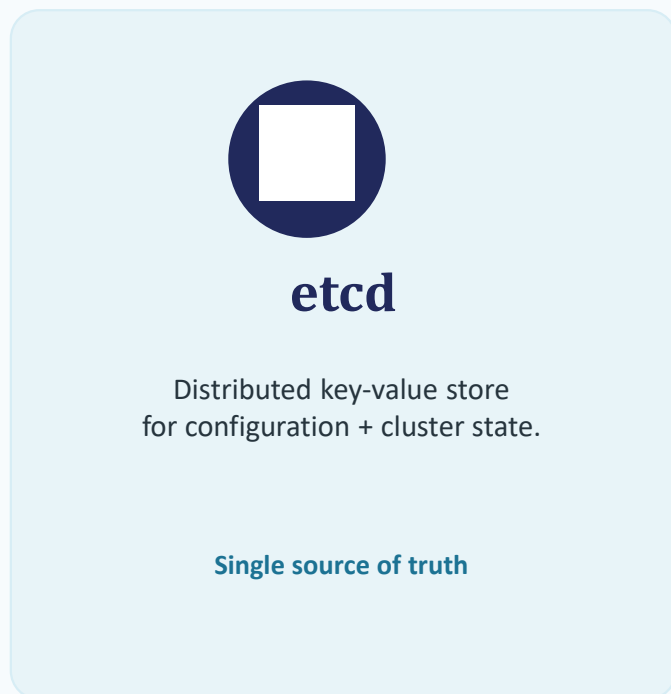
kube-apiserver — the front door



ALL ADMIN TRAFFIC PASSES HERE

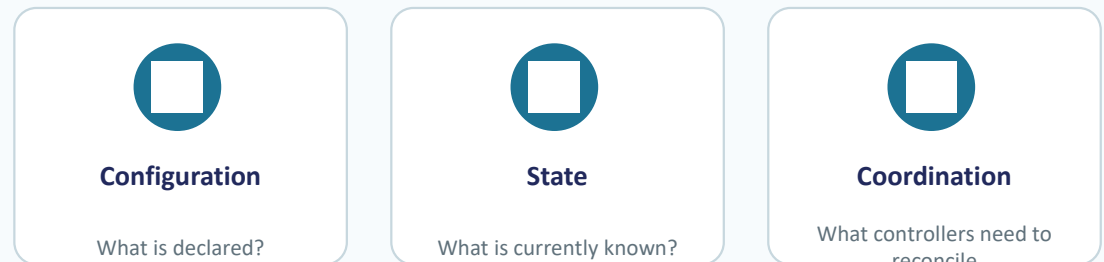


etcd — the cluster state store



MENTAL MODEL

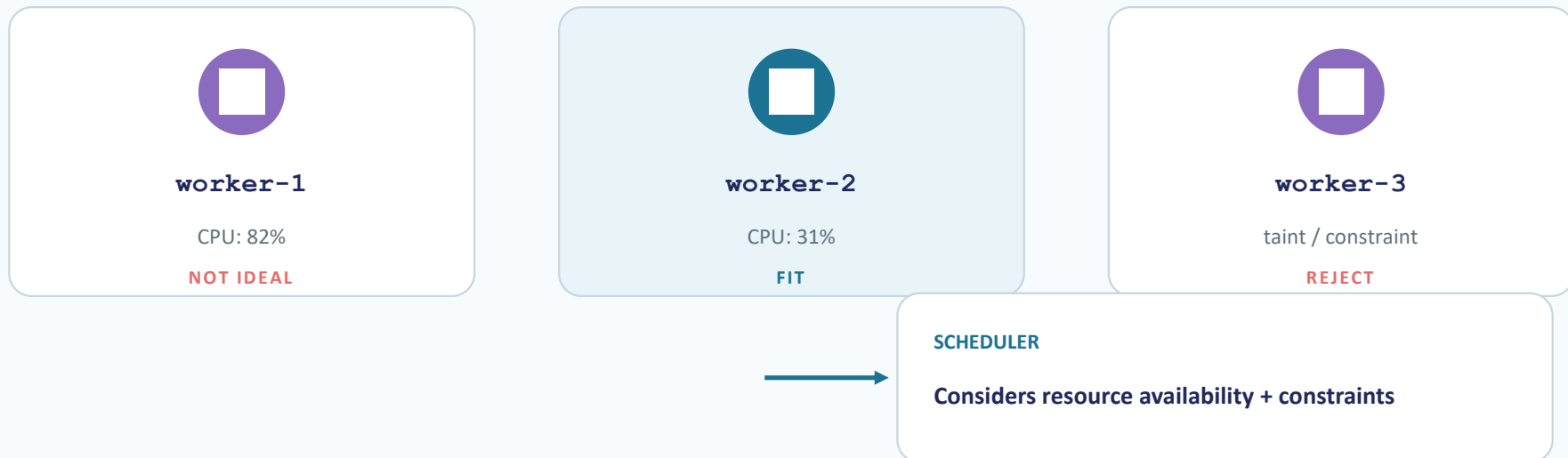
“What should the cluster remember?”



API server is the normal access path

kube-scheduler — choosing a node

A workload needs a home. The scheduler selects a suitable node.



Source: “most suitable node” — not simply “first node”.



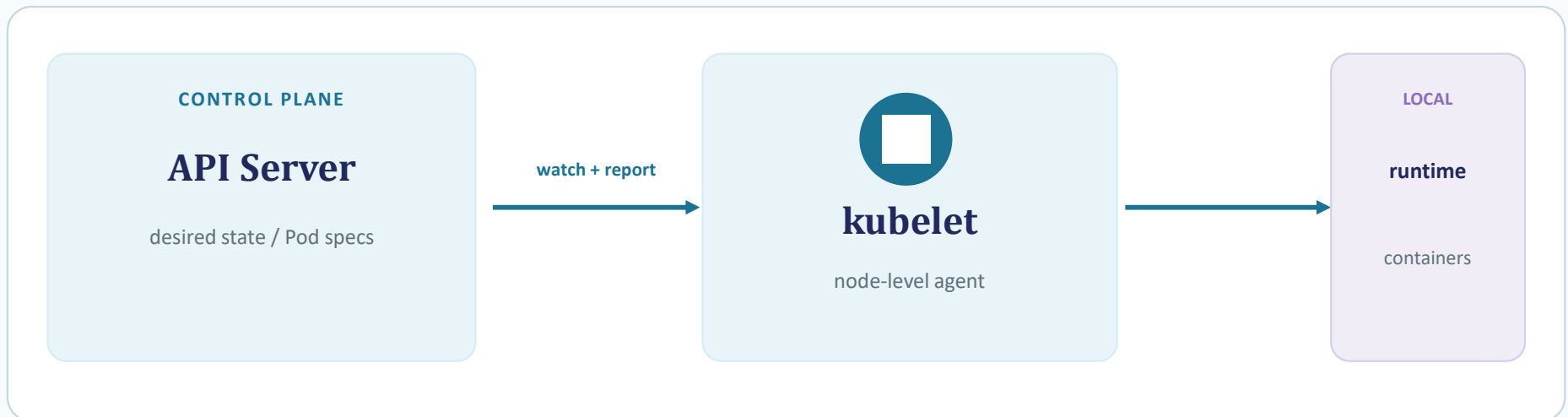
kube-controller-manager — reconciliation loops

Controllers continuously compare desired state with observed state — then act to close the gap.



kubelet — the node agent

Every worker node has a kubelet. It bridges control-plane intent and local container execution.

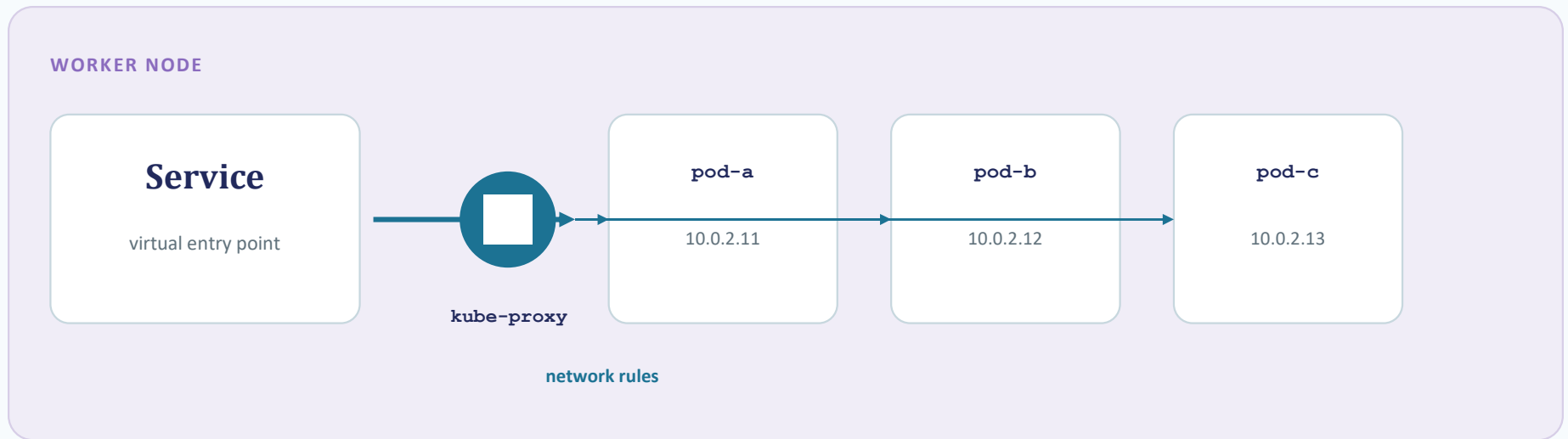


Source framing: kubelet ensures containers are running as defined by Pod specifications.



kube-proxy — node-level service routing

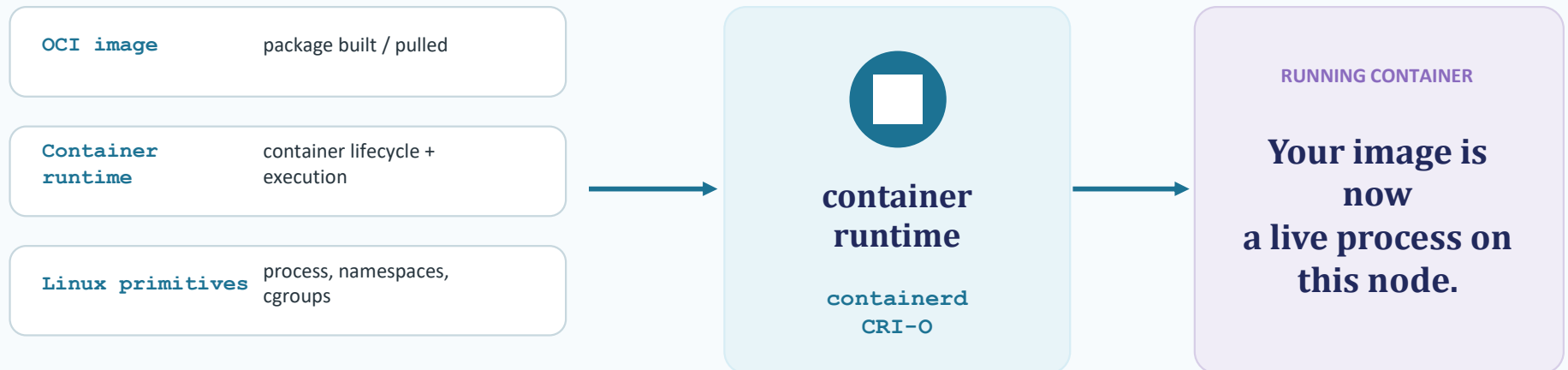
kube-proxy manages network rules on each worker node so Service traffic can reach the appropriate backends.



Routing + load balancing across Service backends

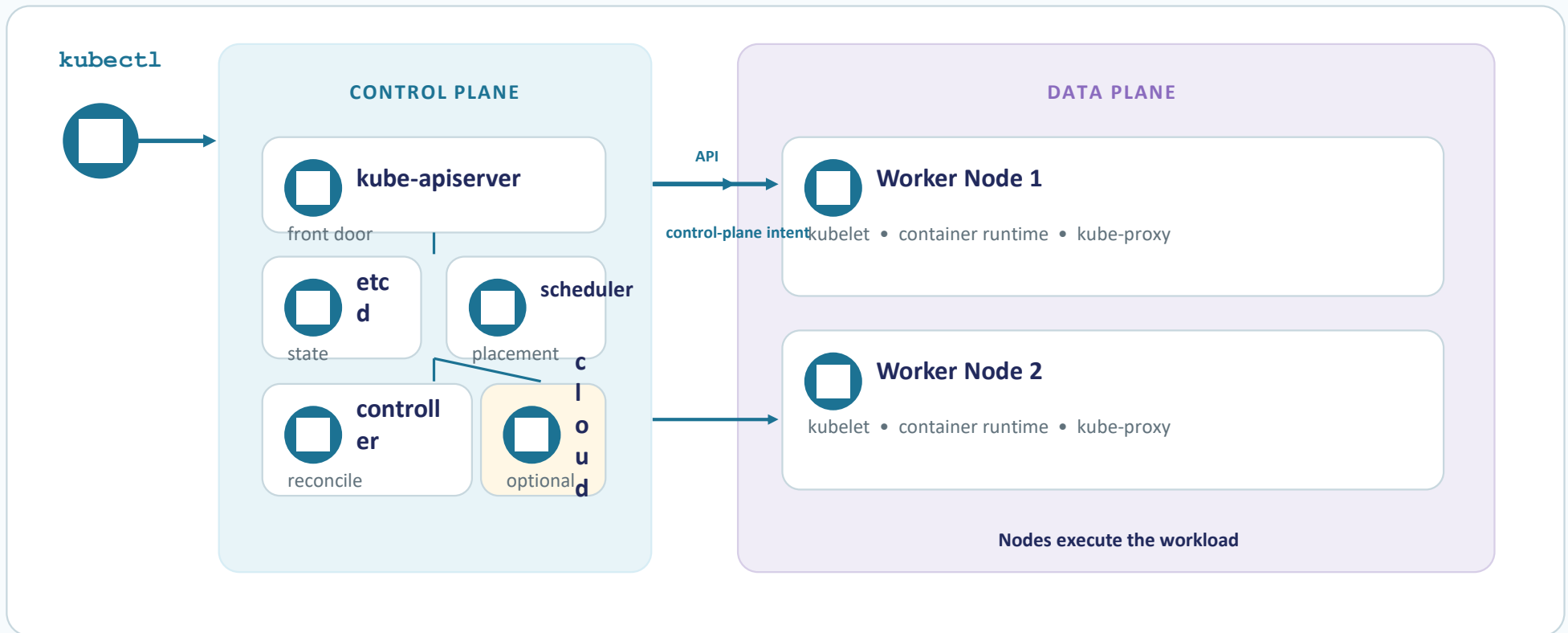
Container runtime — where images become processes

Module 1 taught the image. Kubernetes still needs a runtime on each worker node to actually run the container.



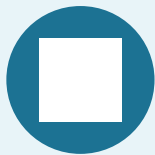
2026 note: Docker Engine is still useful for local development; Kubernetes clusters use a CRI-compatible runtime.

The full cluster — one cohesive system





What is kubectl?



kubectl

A command-line client —
not the Kubernetes cluster.

It sends API requests.

CLIENT

```
$ kubectl get nodes
```

local process

CLUSTER

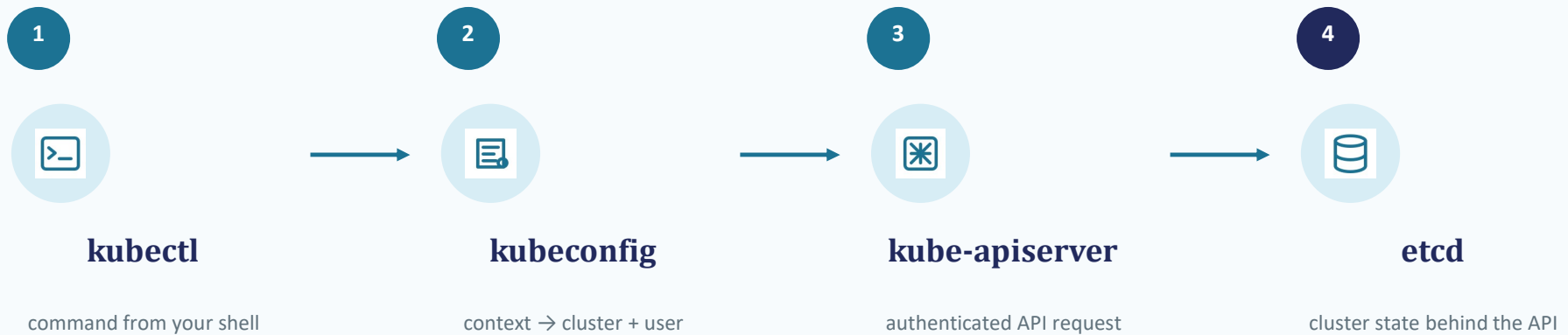
kube-apiserver

API endpoint inside cluster



The cluster does the work; kubectl is the interface.

How kubectl reaches the cluster



kubeconfig does not execute the cluster — it tells kubectl which cluster/context and credentials to use.



Anatomy of a kubectl command

```
kubectl [command] [resource-type] [name] [flags]
```

`kubectl`

the client

`get`

the verb / operation

`nodes`

resource type

`node-1`

optional resource name

`-o wide`

optional flags / output

Read commands left → right: identify the action, target, and modifiers.

kubectl command reference

```
$ kubectl version
```

```
Client Version: v1.37.0
Server Version: v1.37.0
```

```
$ kubectl config view
```

```
apiVersion: v1
clusters:
- name: prod
contexts:
- name: prod
current-context: prod
```

```
$ kubectl config get-contexts
```

CURRENT	NAME
*	prod
	staging

```
$ kubectl config use-context prod
```

```
Switched to context "prod".
```

```
$ kubectl cluster-info
```

```
Kubernetes control plane is running at https://10.0.0.10:6443
CoreDNS is running at https://10.0.0.10:6443/api/v1/namespaces/kube-
system/services/kube-dns:dns/proxy
```

Use these first: confirm client/server versions → verify the active context → prove the API endpoint responds.

kubectl productivity — aliases + autocomplete

 \$ Terminal

```
alias k=kubectl
k get nodes
k get pods
```

OUTPUT

NAME	STATUS	ROLES	AGE	VERSION
node-1	Ready	control-plane	8d	v1.37.0
node-2	Ready	<none>	8d	v1.37.0

 \$ Terminal

```
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
kubectl cre<Tab>
```

OUTPUT

```
$ kubectl create
# Tab completion expands the partial command.
```

Needs the bash-completion package. The complete line makes Tab work for the k alias too. Add all three lines to ~/.bashrc to keep them.



kubectl get — inspect cluster resources

Use get for a fast, tabular snapshot. It lists one or many resources.

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
node-1	Ready	control-plane	8d	v1.37.0
node-2	Ready	<none>	8d	v1.37.0

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
api-7c8d	1/1	Running	0	2d
web-2f91	1/1	Running	1	2d

```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/api-7c8d	1/1	Running	0	2d

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/web	ClusterIP	10.96.0.20	<none>
80/TCP	2d		

Tip: namespaced resources default to the current namespace; cluster-scoped resources such as nodes are not namespace-bound.



kubectl describe — deep inspection

When get tells you “what”, describe helps answer “why”.

Terminal

```
kubectl describe node node-1
```

What to scan first

Conditions • Taints • Addresses
Events • capacity / allocatable

OUTPUT

```
Name: node-1
Roles: control-plane
Labels: kubernetes.io/os=linux
Taints: <none>
Conditions:
  Ready: True
  MemoryPressure: False
  DiskPressure: False
Addresses:
  InternalIP: 10.0.0.11
Events: <none>
```



Help + discoverability — self-serve

Use kubectl itself as your reference manual — discover commands, resources, and fields.

```
$ kubectl --help
```

```
Available Commands:
  api-resources
  cluster-info
  describe
  get
  explain
  ...
```

```
$ kubectl api-resources
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED
KIND			
pods	po	v1	true
Pod			
services	svc	v1	true
Service			

```
$ kubectl explain pods
```

```
KIND:      Pod
VERSION:   v1

FIELDS:
  apiVersion  <string>
  kind        <string>
  metadata    <Object>
```

Terminal

```
kubectl explain pods.spec.containers
kubectl api-resources --sort-by=name
kubectl --help
```

The goal is not to memorize every flag — know where kubectl can teach you the rest.



Architecture in 2026 — what's new

metrics.k8s.io

v1 in Kubernetes v1.37

The Metrics API reaches v1 after nearly nine years in beta. It backs `kubectl top` and autoscaling. You still need `metrics-server` installed — `kind` and `minikube` don't include it.

```
$ kubectl top nodes
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
node-1	420m	21%	3.1Gi	40%

KYAML

Built into `kubectl` (v1.34+)

A stricter, less ambiguous style of YAML: strings are always quoted and braces make structure explicit. Every KYAML file is still valid YAML.

```
$ kubectl get pods -o kyaml
```

```
{
  apiVersion: "v1",
  kind: "Pod",
  metadata: {
    name: "api-7c8d",
```

2026 focus: stable APIs + safer CLI output, without changing the core control-plane / data-plane mental model.



Try It Yourself

1

Check the endpoint

```
kubectl cluster-info
```

```
Kubernetes control plane is running at ...
```

2

List nodes

```
kubectl get nodes
```

```
lab-control-plane   Ready   ...
```

3

Inspect a node

```
kubectl describe node lab-worker
```

```
Conditions: Ready=True
```

4

Set a shortcut

```
alias k=kubectl
```

```
k get nodes → same table
```

Use the 3-node kind cluster from the next slide (demo.html Steps 2–3). Check your context before blaming the cluster.



Create your lab cluster with kind

Terminal

```
kind create cluster \  
  --config kind-cluster.yaml  
kubectl get nodes
```

OUTPUT

lab-control-plane	Ready	control-plane
lab-worker	Ready	<none>
lab-worker2	Ready	<none>

Terminal

```
docker ps --format '{{.Names}}'
```

OUTPUT

```
lab-control-plane  
lab-worker  
lab-worker2
```

kind = Kubernetes IN Docker. Each node is a container — built on everything you learned in Module 1.



See the control plane running

Control-plane components are ordinary pods in the kube-system namespace. Let's find them.

Terminal

```
kubectl get pods \
  -n kube-system -o wide
```

What to notice

Control plane → lab-control-plane only
kube-proxy + CNI → on every node

OUTPUT (NAME and NODE columns)

NAME	NODE
etcd-lab-control-plane	lab-control-plane
kube-apiserver-lab-control-plane	lab-control-plane
kube-controller-manager-lab-control-plane	lab-control-plane
kube-scheduler-lab-control-plane	lab-control-plane
coredns-559f6c778d-fxk7p	lab-control-plane
kube-proxy-2sqss	lab-control-plane
kube-proxy-fcwxc	lab-worker
kube-proxy-6lh9k	lab-worker2
kindnet-w6wtd	lab-worker



Look inside a worker node

Every node runs a kubelet and a container runtime. Step inside a kind worker and see for yourself.

Terminal

```
docker exec -it \
  lab-worker bash
```

What to notice

kubelet + containerd on every node
No Docker inside — crictl talks CRI

INSIDE THE NODE

```
# systemctl is-active kubelet containerd
active
active
# docker ps
bash: docker: command not found
# crictl ps          (columns trimmed)
STATE   NAME                POD
Running kindnet-cni    kindnet-w6wtd
Running kube-proxy     kube-proxy-fcwxc
```



kubectl is just an API client

Every kubectl command becomes an HTTPS request to kube-apiserver. Only the API server talks to etcd.

```
$ kubectl get nodes -v=6
```

```
"Response" verb="GET"  
url="https://127.0.0.1:38793  
/api/v1/nodes?limit=500"  
status="200 OK"
```

```
$ kubectl get --raw /readyz
```

```
ok  
  
# the API server's own health  
# check - what load balancers  
# and kind use to test it
```

```
$ etcdctl get /registry --prefix
```

```
/registry/minions/lab-control-plane  
/registry/minions/lab-worker  
/registry/minions/lab-worker2  
/registry/namespaces/default  
... 450 keys in total
```

kubectl → kubeconfig → HTTPS → kube-apiserver → etcd. Nodes are stored as “minions” — their original 2014 name.



Contexts in practice

Terminal

```
kubectl config set-context lab-system \
  --cluster kind-lab --user kind-lab \
  --namespace kube-system
kubectl config use-context lab-system
```

OUTPUT

```
Context "lab-system" created.
Switched to context "lab-system".
```

Terminal

```
kubectl config get-contexts
kubectl get pods      # no -n needed
kubectl config use-context kind-lab
```

OUTPUT

CURRENT	NAME	NAMESPACE
	kind-lab	
*	lab-system	kube-system

A context = cluster + user + default namespace. Check it before every command that changes something.



Output formats — same data, your way

Same data, your format: a table for people, one field for scripts, the full object for tools.

```
$ kubectl get nodes -o wide
```

NAME	...	CONTAINER-RUNTIME
lab-control-plane		containerd://2.3.4
lab-worker		containerd://2.3.4
lab-worker2		containerd://2.3.4

```
$ kubectl get nodes -o jsonpath=...
```

```
'{.items[*].metadata.name}'
lab-control-plane lab-worker
lab-worker2
```

```
$ kubectl get no lab-worker -o yaml
```

```
{
  apiVersion: "v1",
  kind: "Node",
  metadata: {
    annotations: {
      ...
    }
  }
}
```

Terminal

```
kubectl get nodes -o custom-columns=\
'NAME:.metadata.name,RUNTIME:.status.nodeInfo.containerRuntimeVersion'
```

Humans: -o wide • Scripts: -o jsonpath / -o name • Tools: -o yaml / json / yaml



Watch the architecture work

Create a Deployment, then read the events: each line is a different component doing its job.

Terminal

```
kubectl create deploy web \
  --image=nginx:1.28-alpine \
  --replicas=3
```

Read it top to bottom

controllers create → scheduler places
→ kubelet runs it via containerd

kubectl get events — WHO DID WHAT

WHO	WHAT	OBJECT
deployment-controller	ScalingReplicaSet	web
replicaset-controller	SuccessfulCreate	web-64fc786fb9
default-scheduler	Scheduled	web-...-qh2jw
kubelet	Pulled	web-...-qh2jw
kubelet	Created	web-...-qh2jw
kubelet	Started	web-...-qh2jw



Self-healing and scheduling, live

Desired state always wins. Break something on purpose and watch the control plane fix it.

```
$ kubectl delete pod web-...-6xcqm
```

```
$ kubectl get pods -o wide
web-...-qh2jw   Running   18s   lab-worker2
web-...-qmgwr   Running   4s    lab-worker
web-...-r6bqr   Running   18s   lab-worker

# replaced in 4 seconds
```

```
$ kubectl cordon lab-worker2
```

```
$ kubectl get nodes
lab-control-plane   Ready
lab-worker           Ready
lab-worker2         Ready,
                    SchedulingDisabled
```

```
$ k scale deploy web --replicas=6
```

```
NAME           NODE
web-...-2s57v   lab-worker
web-...-45hg5   lab-worker
web-...-vggt1   lab-worker
web-...-qh2jw   lab-worker2 (old)
# new pods avoid lab-worker2
```

Controller manager replaces lost pods • scheduler skips cordoned nodes • run `kubectl uncordon lab-worker2` when done

Under the hood — CCM, kube-proxy modes, cgroups



cloud-controller-manager

The fifth control-plane component on cloud clusters (EKS, GKE, AKS). It talks to the cloud API: builds load balancers for LoadBalancer Services, sets node addresses, removes nodes whose VM is gone.

kind has no cloud, so there is none. In Module 4B, cloud-provider-kind plays that role.



kube-proxy: iptables, IPVS, nftables

kube-proxy turns Services into packet rules on every node. iptables is still the default — our cluster uses it.

nftables mode is stable since v1.33 and scales better on large clusters; IPVS is the older alternative. Some network plugins (e.g. Cilium) replace kube-proxy entirely.

2026: kubelet refuses cgroup v1 by default since v1.35 (use cgroup v2). Images: registry.k8s.io — k8s.gcr.io is frozen.



Lab: look under the hood

Terminal

```
kubectl -n kube-system get configmap kube-proxy \
-o jsonpath='{.data.config\.conf}' | grep ^mode
```

OUTPUT

```
mode: iptables
# the default — nftables and ipvs are the alternatives
```

Terminal

```
docker info --format '{{.CgroupVersion}}'
kubectl get --raw /api/v1/nodes/lab-worker/proxy/configz \
| grep -o '"cgroupDriver":"[a-z]*"'
```

OUTPUT

```
2                                     ← cgroup v2 host
"cgroupDriver":"systemd"             ← same as containerd
```

Terminal

```
kubectl get pods -n kube-system -o jsonpath=\
'{range .items[*]}{.spec.containers[0].image}{"\n"}{end}'
```

OUTPUT

```
registry.k8s.io/kube-apiserver:v1.37.0
registry.k8s.io/kube-proxy:v1.37.0 ...
# no cloud-controller-manager on kind
```



Common pitfalls — diagnose the path first



Wrong context selected

```
kubectl config get-contexts  
→ kubectl config use-context  
   <name>
```

You may be talking to the wrong cluster.



Connection refused

```
kubectl config view  
→ verify server endpoint +  
   credentials
```

The API endpoint in kubeconfig may be wrong or unreachable.



Resource not found

```
kubectl api-resources  
→ check resource type / scope
```

You may be using the wrong resource name or namespace.



“Where is my workload?”

```
kubectl get ... -A  
→ compare namespaces
```

Namespaced resources are scoped to the current namespace by default.



Module recap — the mental model



Kubernetes

Orchestrates containers at production scale.



Control plane

API server, etcd, scheduler, controllers.



Data plane

kubelet, runtime, kube-proxy on workers.



kubectl

Client that talks to the API server.



kubeconfig

Context + cluster + credentials for kubectl.



CLI workflow

get → describe → explain → troubleshoot.

If you can draw the request path, you can troubleshoot the cluster path.



KNOWLEDGE CHECK

9 questions

Before you move on

Real scenarios from running clusters: architecture, kubectl, contexts and troubleshooting. Answer key on the last quiz slide.

CONTROL
PLANE

decides

DATA PLANE

executes

KUBECTL

talks to API



Question 1 of 9

Your teammate says the app is running, but kubectl shows nothing. What is the most likely reason?

Terminal

```
$ kubectl get pods
No resources found in
default namespace.
```

Think about

Where is kubectl looking?
Namespace + context

CHOOSE ONE

- A. The API server is down
- B. You are looking in the wrong namespace or context
- C. etcd lost the cluster data
- D. The kubelet on every node crashed



Question 2 of 9

You create a Deployment with 3 replicas. Which component decides which node each new pod runs on?

Terminal

```
$ kubectl create deploy web \
  --image=nginx --replicas=3
deployment.apps/web created
```

Think about

Who places the pod,
and who runs it?

CHOOSE ONE

- A. kubelet
- B. kube-proxy
- C. kube-scheduler
- D. etcd



Question 3 of 9

A pod from a Deployment is deleted, and a replacement appears 4 seconds later. Which component made that happen?

Terminal

```
$ kubectl delete pod web-...-6xcqm
$ kubectl get pods
web-...-qmgwr Running 4s
```

Think about

Desired state vs
current state

CHOOSE ONE

- A. The controller manager — its ReplicaSet controller saw 2 of 3 pods and created one
- B. The kubelet — it restarts any pod that is deleted
- C. The scheduler — it recreates the pods it placed
- D. etcd — it restores deleted objects automatically



Question 4 of 9

kubectl suddenly fails with the error below. What should you check first?

Terminal

```
$ kubectl get nodes
The connection to the server
127.0.0.1:6443 was refused -
did you specify the right
host or port?
```

Think about

Follow the path: kubectl →
kubeconfig → API server

CHOOSE ONE

- A. Reinstall kubectl
- B. Delete and recreate etcd
- C. Restart every pod in kube-system
- D. Your kubeconfig context and server address — and that the cluster is actually running



Question 5 of 9

Where does Kubernetes store the desired state you just applied, and how do components reach it?

Terminal

```
$ kubectl apply -f web.yaml  
deployment.apps/web created
```

Think about

The cluster's single
source of truth

CHOOSE ONE

- A. On each kubelet's local disk — every node keeps its own copy
- B. In etcd — and every component goes through the API server to reach it
- C. Inside your kubeconfig file
- D. Inside the container image



Question 6 of 9

A developer says: “Our cluster uses containerd, so it can’t run images built with Docker.” Is that right?

Terminal

```
$ kubectl get nodes -o wide
...   CONTAINER-RUNTIME
...   containerd://2.3.4
```

Think about

OCI images + CRI runtimes
(Module 1 and slide 11)

CHOOSE ONE

- A. Yes — rebuild every image with containerd
- B. Yes — install Docker on every node first
- C. Partly — convert the images with kubectl first
- D. No — Docker builds standard OCI images, and containerd runs OCI images



Question 7 of 9

Your kubeconfig has prod and staging contexts. What is the safest habit before running `kubectl delete`?

Terminal

```
$ kubectl config get-contexts
CURRENT  NAME
*        prod
         staging
```

Think about

What is `kubectl` talking to right now?

CHOOSE ONE

- A. Confirm the active context first (`get-contexts` or `current-context`)
- B. Always add `--force`
- C. Delete your kubeconfig after each session
- D. Run the command twice to be sure



Question 8 of 9

A CI script needs only the node names, on one line, with no table headers. Which command fits best?

Terminal

```
$ kubectl get nodes
NAME          STATUS    ROLES ...
lab-worker    Ready    <none> ...
```

Think about

Machine-readable
output

CHOOSE ONE

- A. kubectl describe nodes
- B. kubectl get nodes -o wide
- C. kubectl get nodes -o jsonpath='{.items[*].metadata.name}'
- D. kubectl get all



Question 9 of 9

Service traffic works on every node except lab-worker2. Which component on that node do you suspect first?

Terminal

```
$ kubectl get pods -n kube-system \
-o wide | grep lab-worker2
kube-proxy-6lh9k    CrashLoopBackOff
kindnet-svl5w      Running
```

Think about

Node-level Service routing
(slide 10)

CHOOSE ONE

- A. etcd
- B. kube-proxy — it programs the node's Service routing rules
- C. kube-scheduler
- D. kube-controller-manager



Answer key

Check your answers. Each one maps back to a slide and a hands-on lab step.

\$ Questions 1 - 3

- Q1 B wrong namespace or context
→ slide 31 • lab Step 7
- Q2 C kube-scheduler places pods
→ slide 7 • lab Step 11
- Q3 A controller manager replaces
→ slide 8 • lab Step 12

\$ Questions 4 - 6

- Q4 D check kubeconfig + endpoint
→ slide 31 • lab Step 7
- Q5 B etcd, via the API server
→ slide 6 • lab Step 6
- Q6 D OCI images run anywhere
→ slide 11 • lab Step 5

\$ Questions 7 - 9

- Q7 A confirm the context first
→ slide 14 • lab Step 7
- Q8 C -o jsonpath for scripts
→ slide 15 • lab Step 8
- Q9 B kube-proxy routes Services
→ slide 10 • lab Step 4

● ● ● Score guide

8-9: ready for Module 3 • 6-7: re-read the slides you missed
5 or fewer: redo lab Steps 4-13, then try the quiz again

A wrong answer is useful — rerun that lab step until the output makes sense.



MODULE 2 COMPLETE

Next: Module 3

Pods, Pod Lifecycle & Services

You now know where Kubernetes decisions happen, where containers run, and how kubectl enters the system.

